Vernex Clinic OS: Integration Guide

How to connect WhatsApp, SMS, online payments and the other services a clinic needs to go live.

Written for the product owner. Technical steps are marked **[Developer]**; things only you can do (accounts, documents, payments) are marked **[You]**.

---

1. Where things stand today

Service	What the app does now	What is still needed
WhatsApp	Every message the app creates is saved in a queue. A sender (`whatsapp-dispatch`) is built and tested. It runs in **simulator mode**: nothing leaves the system, messages are marked "delivered (simulated)". Patient opt-outs are respected.	Your Meta business account, a WhatsApp number, approved message templates, then switch the provider on (Section 3). Receiving patient replies and live delivery ticks needs the inbound webhook, which is **not built yet**.
SMS	Not built.	A DLT registration (India), an SMS provider account, and a sender function modelled on the WhatsApp one (Section 4).
Online payments	Not built. The booking page shows "Pay online: not available yet". The bill screens record payments by hand (cash, UPI, card).	A Razorpay account, plus the order, verification and webhook functions (Section 5).
Email	Not built. Staff sign in with email and password only.	Optional. An email provider for receipts and password reset (Section 6).

Honest summary: WhatsApp is ready to switch on once you have the accounts. SMS and online payments need build work as well as accounts. Nothing here requires rewriting the app. Each service plugs into an existing queue or screen.

---

2. What to prepare first (the slow parts)

Start these now. They depend on other people and take days or weeks, not hours.

1. **A legal entity and documents [You].** Company registration, PAN, GST number (if registered), a bank account, and a website with a privacy policy and terms. Meta, Razorpay and the SMS registry all ask for these.
2. **A dedicated phone number for WhatsApp [You].** It must not be registered on the normal WhatsApp or WhatsApp Business app. It must receive one verification call or SMS.
3. **Domain name and production website [You + Developer].** Meta and Razorpay both review the website.
4. **DLT registration for SMS [You].** Required by Indian telecom rules before any business SMS is delivered (Section 4). This is usually the longest step.

---

3. WhatsApp (Meta WhatsApp Business Platform, Cloud API)

3.1 Accounts [You]

1. Create a **Meta Business account** at business.facebook.com and complete **Business Verification** (upload company documents). Without it, you are limited to very low message volumes.
2. In Meta for Developers, create an **App** of type *Business* and add the **WhatsApp** product.
3. Add your phone number to the WhatsApp Business Account and verify it. Note two values from the API Setup page: the **Phone Number ID** and the **WhatsApp Business Account ID**.

4. Create a **System User** in Business Settings, give it access to your app and WhatsApp account, and generate a **permanent access token** with the permissions `whatsapp_business_messaging` and `whatsapp_business_management`. (The temporary token on the setup page expires in about a day and is only for testing.)

3.2 Message templates [You]

WhatsApp only allows free-form messages within 24 hours after the patient last wrote to you. Everything else (appointment confirmations, reminders, prescriptions, bills) must use a **template approved by Meta**.

1. In WhatsApp Manager, create templates. Use category **Utility** for appointment, prescription and billing messages. Reminders to come back may be treated as Marketing. Check the current rules and prices with Meta.
2. Wait for approval (usually minutes to a couple of days).
3. In the app, open **WhatsApp** → **Templates**, create a matching template and put Meta's template name in the **provider template id** field.
4. Variables: the app sends a list of values in the same order as the `{{1}}, {{2}}` placeholders in your template.

3.3 Turn it on [Developer]

Set these secrets on the Supabase project, then redeploy nothing. Secrets take effect on the next call.

```
supabase secrets set WHATSAPP_PROVIDER=meta_cloud
supabase secrets set WHATSAPP_ACCESS_TOKEN=<permanent token>
supabase secrets set WHATSAPP_PHONE_NUMBER_ID=<phone number id>
```

Test with one message to your own phone, then check that the message shows as **sent** (not "simulated") in WhatsApp → Conversations. To go back to safe testing at any time, set `WHATSAPP_PROVIDER=simulator`.

3.4 Still to build for full WhatsApp [Developer]

- **Webhook function** to receive delivery receipts (sent, delivered, read), patient replies and opt-outs ("STOP"). In Meta's app settings you then set the **Callback URL** to the function address and a **Verify token**, and subscribe to the `messages` field. The app already has tables for webhook events, conversations and consents; only the receiving function is missing.
- **One number per clinic or one per brand.** Today there is a single set of credentials for the whole system. If every clinic should send from its own number, credentials must be stored per clinic. Decide this before launch.

3.5 Rules to remember

- Get patient **opt-in** (the booking form is a good place). The app honours opt-outs and will refuse to message those patients.
- Meta charges per conversation or per template message, depending on category and country. Check current rates before you price the product.
- A poor quality rating (many blocks and reports) lowers your sending limit. Do not send promotions to people who did not ask.

---

4. SMS (India)

4.1 Why it is needed

- **Appointment reminders** for patients without WhatsApp.
- **One-time passwords (OTP)** for the "My appointments" lookup. Right now anyone who knows a patient's phone number can confirm that patient has an appointment at a clinic. The security review recommends an OTP once an SMS gateway exists.

4.2 DLT registration [You]

Indian operators block business SMS that is not registered on **DLT** (Distributed Ledger Technology), operated separately by each operator (Jio, Airtel, Vi, BSNL).

1. Register your company as a **Principal Entity** on one operator's DLT portal and get an **Entity ID**.
2. Register your **sender ID (header)**, a six-letter name such as `VERNXC`.
3. Register each **message template** (the exact wording, with variables) and get a **Template ID** for each. Unregistered wording is rejected.
4. Your SMS provider usually helps with this and links your registration.

4.3 Provider [You]

Pick one Indian SMS provider (for example MSG91, Fast2SMS, or Twilio). Compare price per SMS, delivery reports, and support. You will receive an **auth key** and need to give them your Entity ID, sender ID and Template IDs.

4.4 Build [Developer]

Not built. The plan mirrors WhatsApp, about one to two days of work:

1. A `sms_messages` queue table (phone, text, status, DLT template id, provider message id).
2. An `sms-dispatch` function with the same simulator-first design and one adapter for the chosen provider.
3. Provider secrets: `SMS_PROVIDER`, `SMS_AUTH_KEY`, `SMS_SENDER_ID`, `SMS_ENTITY_ID`.
4. An OTP step on the My appointments page, with expiry and rate limits.

---

5. Online payments (Razorpay)

5.1 Account [You]

1. Sign up at razorpay.com and complete **KYC** (business PAN, bank account, website). Activation can take several days.
2. Use **Test mode** first. It gives you a **Key ID** and **Key Secret** that never move real money.
3. Later, generate **Live mode** keys. Turn on UPI, cards and netbanking as needed.
4. Create a **Webhook** in the dashboard pointing to the app's payment webhook address (once built), choose events `payment.captured`, `payment.failed` and `refund.processed`, and choose a **webhook secret**.

5.2 How the payment flow works

1. The patient (or receptionist) chooses **Pay online** for a booking or a bill.
2. The server creates a Razorpay **Order** for the amount, using the Key Secret. The secret never reaches the browser.
3. The browser opens Razorpay's checkout using only the **Key ID**.
4. After payment, the server **verifies the signature** (an HMAC of order id and payment id with the Key Secret). Only then is the booking or bill marked paid.
5. The webhook is a safety net: if the patient closes the browser after paying, the webhook still marks it paid.

5.3 Build [Developer]

Not built. Today the Key ID setting in the frontend is a placeholder and is not read anywhere.

1. An `online_payments` table linking each Razorpay order to a booking or invoice, with status and the gateway's ids.
2. Functions `create-payment-order`, `verify-payment` and `payment-webhook`.

3. Enable the **Pay online** option in the booking flow and the bill screens, and reconcile with the existing manual payment records and refunds.
4. Secrets: `PAYMENT_PROVIDER` (`simulator` or `razorpay`), `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`.

5.4 Decisions for you

- **Whose bank account gets the money?** Each clinic with its own Razorpay account (simplest legally, more setup), or one Vernex account that pays clinics out (needs Razorpay Route and contracts, and tax handling).
- **Refund policy** and who approves refunds.
- **GST invoicing** on platform fees.

---

6. Other things to arrange before launch

Area	What to do
Production database [Developer]	Create a separate production Supabase project (the current one is for development and will be thrown away). Choose the Mumbai region for latency and data residency. Turn on daily backups and point-in-time recovery. Build it from the migration files, not by copying.
Hosting [Developer]	Deploy the frontend (Vercel configuration is already in the project) on your domain with HTTPS.
Email [You + Developer]	Staff password-reset emails come from Supabase's default sender, which is rate-limited and not suitable for production. Add your own SMTP provider (for example Amazon SES or Resend) in Supabase, and verify your domain (SPF and DKIM records).
Monitoring [Developer]	Add error tracking (for example Sentry) and uptime checks.
Privacy and law [You]	Health data is sensitive personal data under India's **Digital Personal Data Protection Act, 2023**. Get a privacy policy, patient consent wording, a data-retention policy and a grievance contact reviewed by a lawyer. Sign a data processing agreement with each clinic.
Security [Developer]	Rotate every development key before production (service role key, access tokens, database password). Never reuse development secrets. Keep service keys only in Supabase secrets, never in the frontend.
Support [You]	A support email or phone and a plan for who responds when a clinic cannot book patients.

---

7. Suggested order

Step	Who	Typical time
1. Start Meta Business verification, DLT registration, Razorpay KYC in parallel	You	Start now. Wait 1 to 3 weeks
2. Switch WhatsApp from simulator to Meta (Section 3.3) once approved	Developer	Half a day
3. Build WhatsApp webhook	Developer	1 to 2 days

Step	Who	Typical time
4. Build SMS queue, sender and OTP	Developer	1 to 2 days
5. Build Razorpay order, verify and webhook, enable Pay online	Developer	3 to 5 days
6. Production project, domain, email, backups, monitoring	Developer	1 to 2 days
7. Pilot with one real clinic, fix what they find	Both	2 to 4 weeks

---

8. Secrets reference

Set all of these only in Supabase (`supabase secrets set NAME=value`), never in the frontend or in code.

Name	Used for	Status
`WHATSAPP_PROVIDER`	`simulator` (default) or `meta_cloud`	In use
`WHATSAPP_ACCESS_TOKEN`	Meta permanent token	In use
`WHATSAPP_PHONE_NUMBER_ID`	Meta sending number	In use
`WHATSAPP_GRAPH_VERSION`	Meta API version (default `v21.0`)	Optional
`WHATSAPP_VERIFY_TOKEN`, `WHATSAPP_APP_SECRET`	Inbound webhook	When webhook is built
`SMS_PROVIDER`, `SMS_AUTH_KEY`, `SMS_SENDER_ID`, `SMS_ENTITY_ID`	SMS	When SMS is built
`PAYMENT_PROVIDER`, `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`	Payments	When payments are built

Provider requirements, prices and approval times change. Treat this as a map and confirm each step against the provider's current documentation before you commit money or dates.